Rapport

Management avancé des SI

2026-03-23

TERRIEN Swan, TOURAINE Melyna, COUSIN Gautier, MAANLI Naël
IUT de Nantes
2026
Informatique

Sommaire

I. Contexte et acteurs 3
II. Fonctionnalités et technologies 3
III. Explication BPMN 3
IV. Processus métier 4
IV.1. Affichage station Bicloo 4
IV.2. Affichage parking P+R 5
IV.3. Affichage carte Nantes, carte vélo, carte touristique 6
IV.4. Affichage des lignes cyclables 7
IV.5. Accès horaires parking ou autres 8
IV.6. Accès tutoriel utilisation Bicloo 9
IV.7. Consultation alertes travaux 10
IV.8. Consultation itinéraire 11

I. Contexte et acteurs

Contexte — Dans le cadre de la SAE du semestre 4, l'objectif est de concevoir une application orientée micro-services, connectée à des sources OpenData. Une application mobile était prévue au départ ; elle est conservée dans ce rapport comme acteur cible afin de formaliser les processus métier.

Acteurs :

- **Utilisateur** : Utilise l'application pour trouver des infos sur lignes cyclables et parkings
- **API/Proxy** : Interface communiquant avec les données et les sources OpenData
- **Application mobile** : Interface utilisateur
- **API source** : Fournit les données via OpenData Nantes Métropole

II. Fonctionnalités et technologies

Fonctionnalités prioritaires :

- Affichage stations Bicloo
- Affichage parkings P+R
- Affichage cartes
- Affichage lignes cyclables

Fonctionnalités secondaires :

- Horaires
- Tutoriels

Fonctionnalités tertiaires :

- Alertes travaux
- Itinéraires

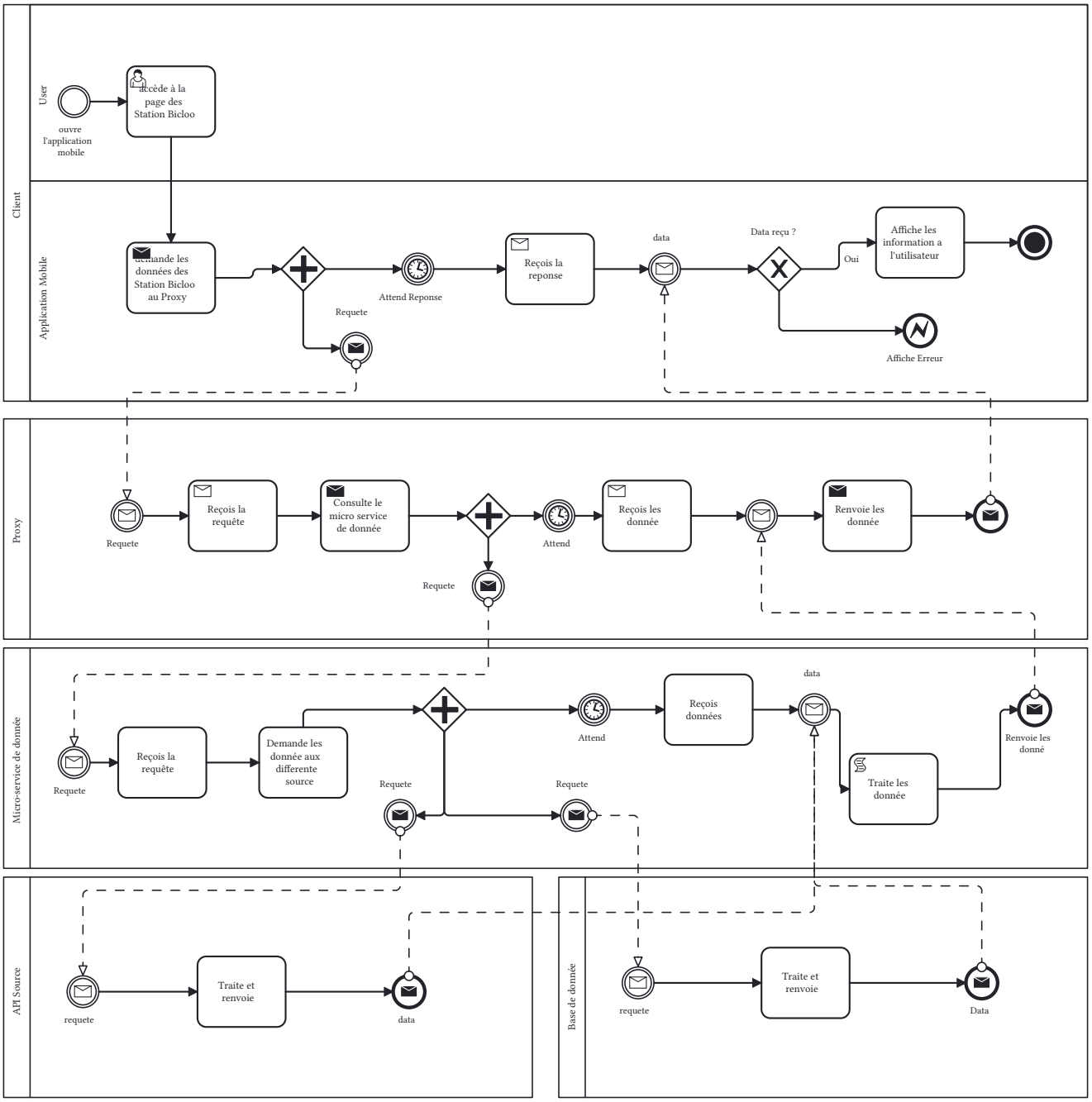
III. Explication BPMN

Nous avons produit un BPMN par fonctionnalité pour clarifier les enchainements entre utilisateur, application mobile, proxy et API sources. Ce choix évite un schéma global trop chargé et rend chaque parcours plus lisible. Le périmètre actuel cible principalement les usages vélo, tout en conservant une architecture extensible vers d'autres modes de transport.

IV. Processus métier

IV.1. Affichage station Bicloo

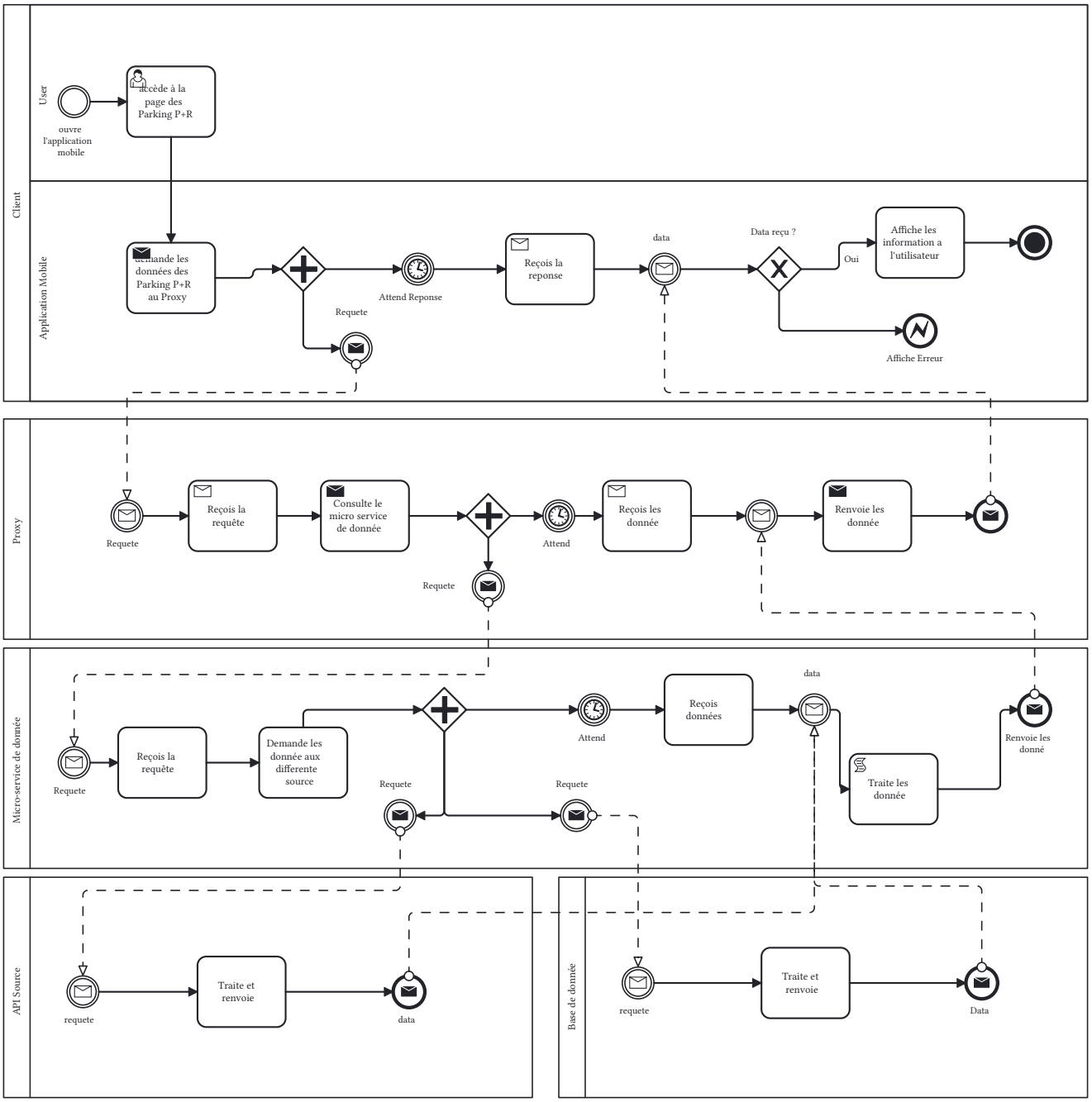
Figure 1 — Affichage information stations Bicloo



Ce processus démarre quand l'utilisateur ouvre l'application puis accède à la page des stations Bicloo. L'application mobile envoie la requête au proxy et se met en attente. Le proxy transmet la demande à un micro-service de données, qui interroge en parallèle l'API source et la base de données, attend les retours, puis traite et renvoie un résultat au proxy. Le proxy renvoie ensuite les données à l'application ; une passerelle vérifie la réception des données: en cas de succès les informations sont affichées, sinon une erreur est présentée.

IV.2. Affichage parking P+R

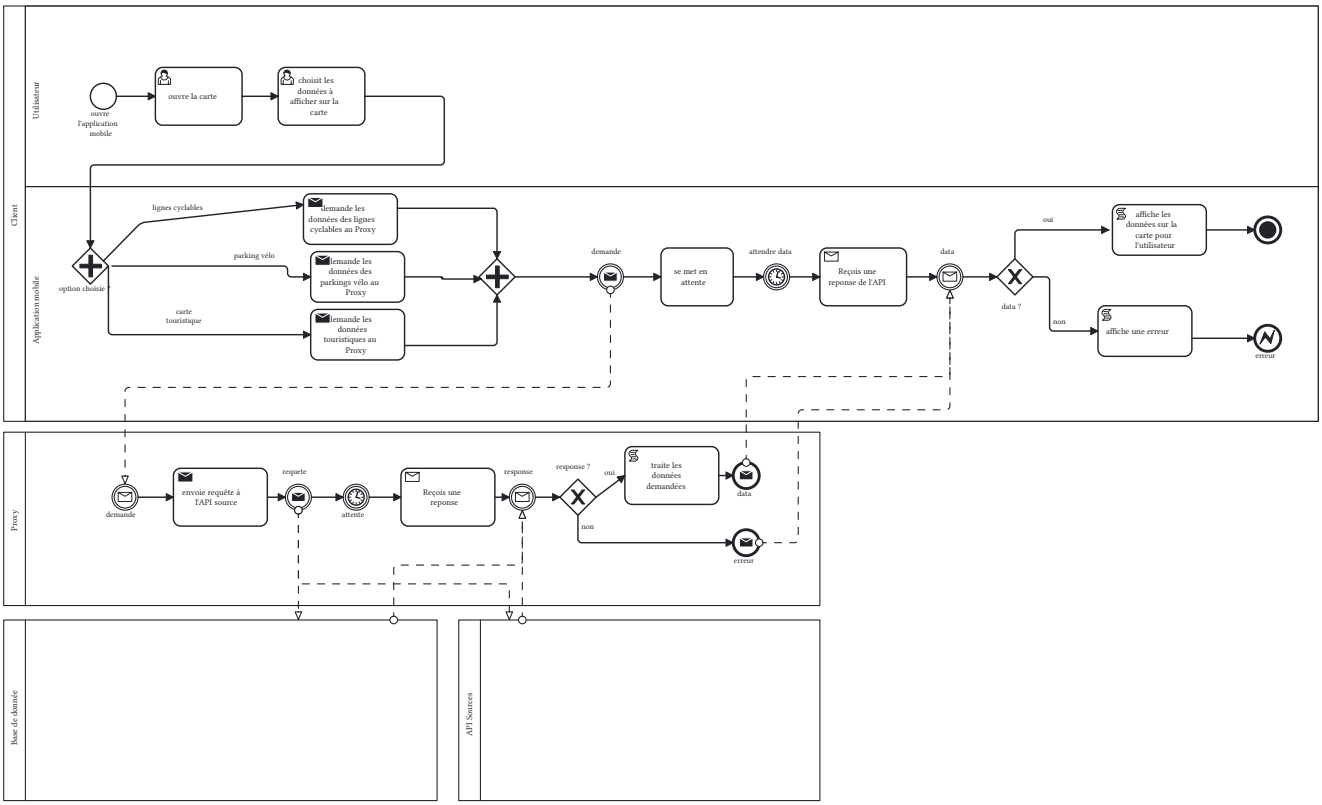
Figure 2 — Affichage informations parkings P+R



Le scénario commence par l'ouverture de l'application puis l'accès à la page des parkings P+R. L'application envoie la demande au proxy et attend la réponse. Le proxy délègue la récupération au micro-service de données, qui envoie des requêtes à l'API source et à la base de données, attend les réponses, traite les informations, puis renvoie les données consolidées au proxy. Le proxy transmet enfin les données à l'application, qui affiche les informations si elles sont reçues, sinon affiche une erreur.

IV.3. Affichage carte Nantes, carte vélo, carte touristique

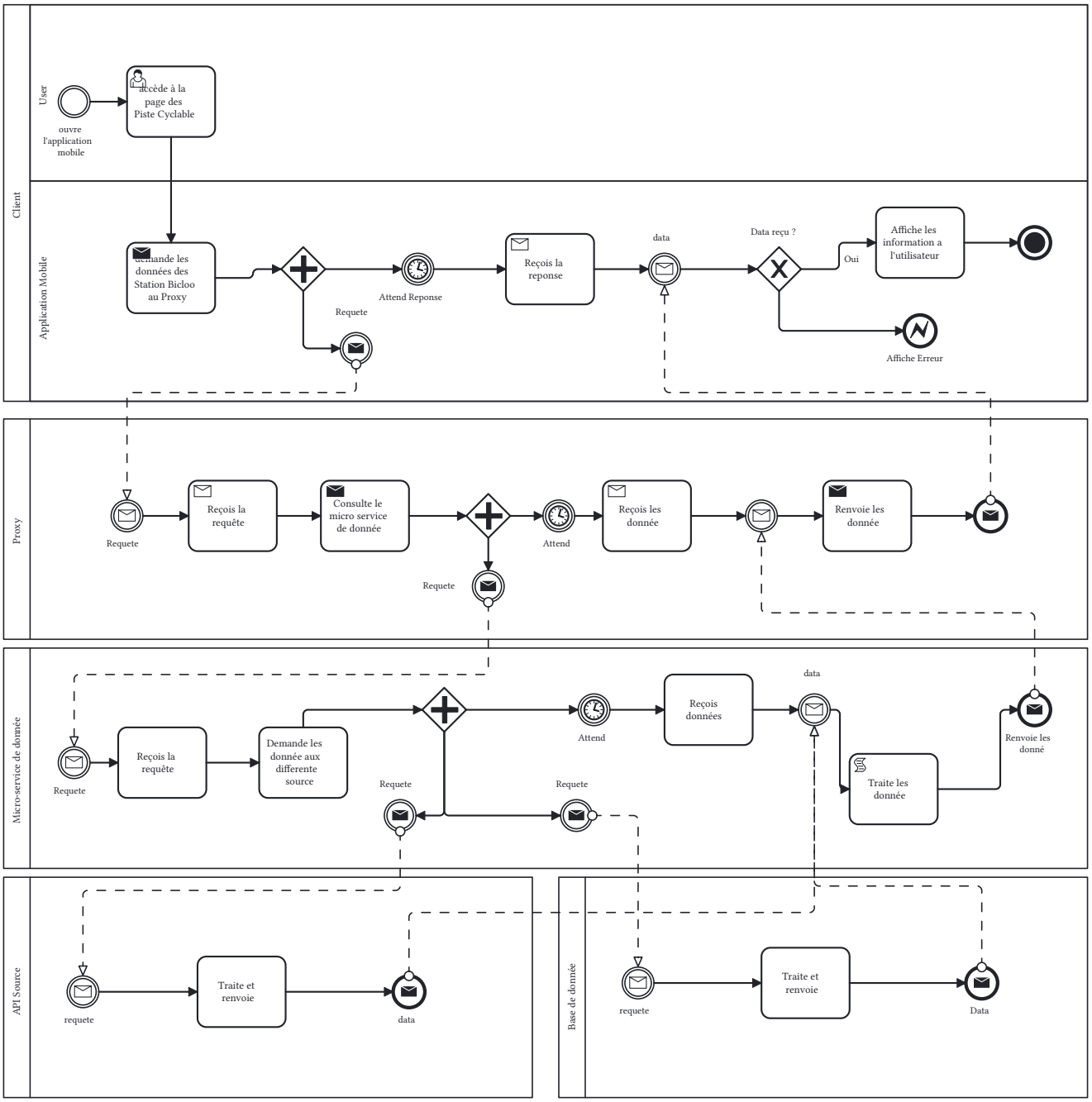
Figure 3 — Consultation cartes



L'utilisateur ouvre la carte et choisit les données à afficher. Le BPMN modélise trois demandes envoyées en parallèle par l'application (lignes cyclables, parkings vélo, carte touristique), puis une synchronisation avant l'envoi d'une requête globale au proxy. Le proxy relaie cette requête vers les sources (API et base de données), attend une réponse, puis vérifie sa validité: si la réponse est exploitable, il traite les données et les renvoie ; sinon, il renvoie une erreur. L'application affiche ensuite les données sur la carte ou un message d'erreur selon le résultat reçu.

IV.4. Affichage des lignes cyclables

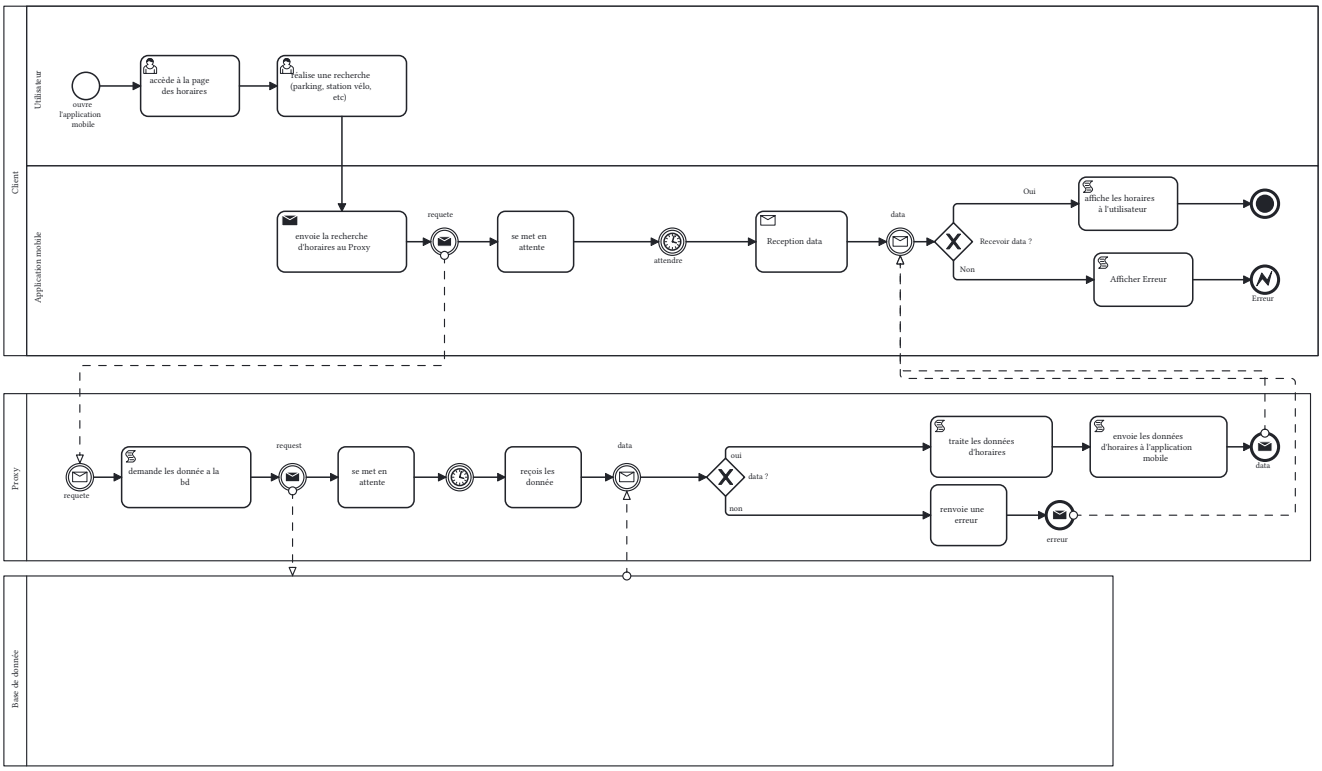
Figure 4 — Affichage des lignes cyclables



Le processus commence lorsque l'utilisateur ouvre l'application puis accède à la page des pistes cyclables. L'application envoie une requête au proxy et attend la réponse. Le proxy consulte un micro-service de données qui interroge les sources, attend les retours, traite les données puis les renvoie au proxy. Le proxy retransmet le résultat à l'application, qui vérifie la présence des données: si elles sont disponibles, elles sont affichées ; sinon, une erreur est affichée.

IV.5. Accès horaires parking ou autres

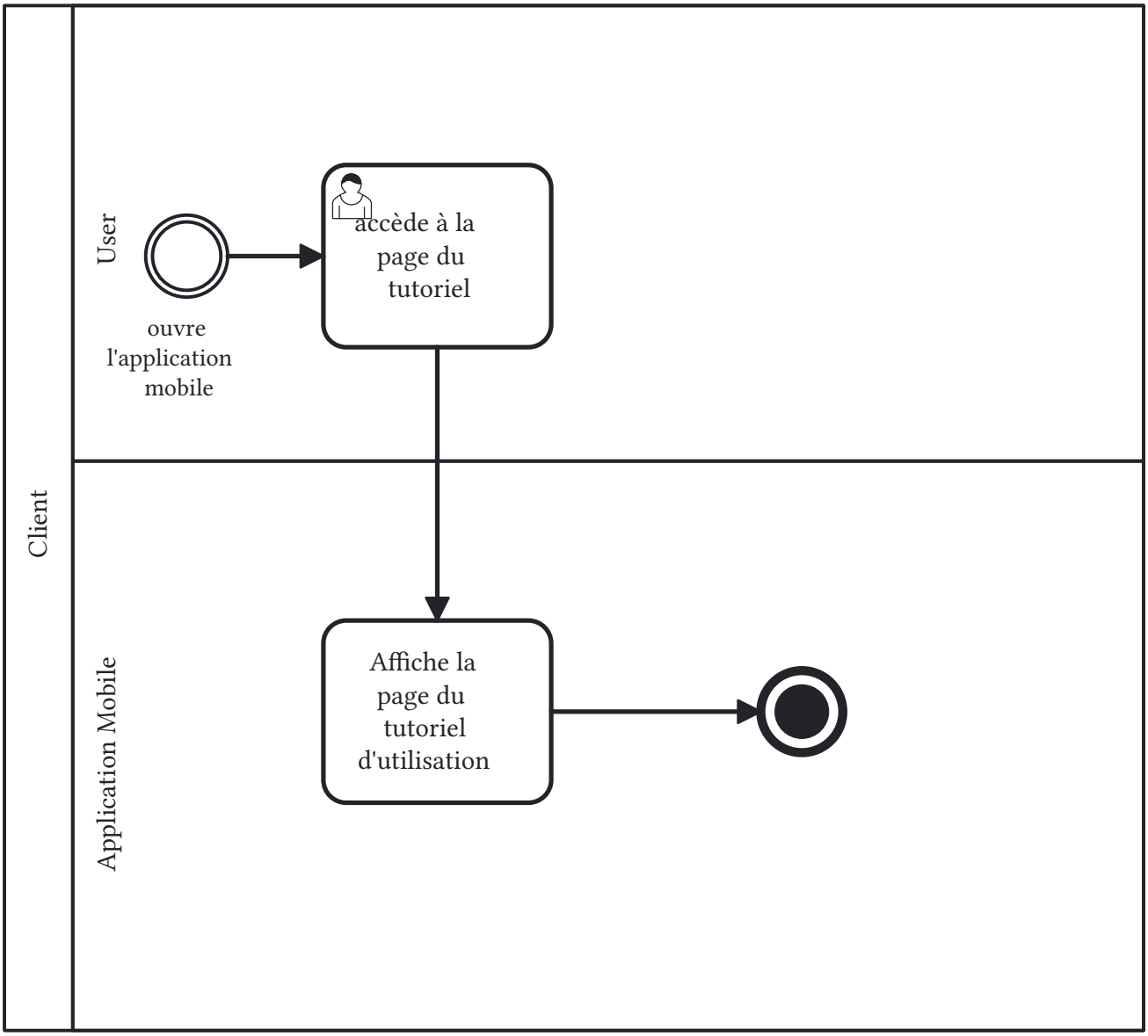
Figure 5 — Affichage horaires



L'utilisateur ouvre la page des horaires puis lance une recherche (parking, station vélo, etc.). L'application envoie la requête au proxy et passe en attente. Le proxy demande les données d'horaires à la base de données, se met en attente, puis reçoit la réponse. Une décision est ensuite prise: si des données sont disponibles, le proxy les traite et les renvoie à l'application ; sinon, il renvoie une erreur. Côté application, les horaires sont affichés en cas de succès, sinon un message d'erreur apparaît.

IV.6. Accès tutoriel utilisation Bicloo

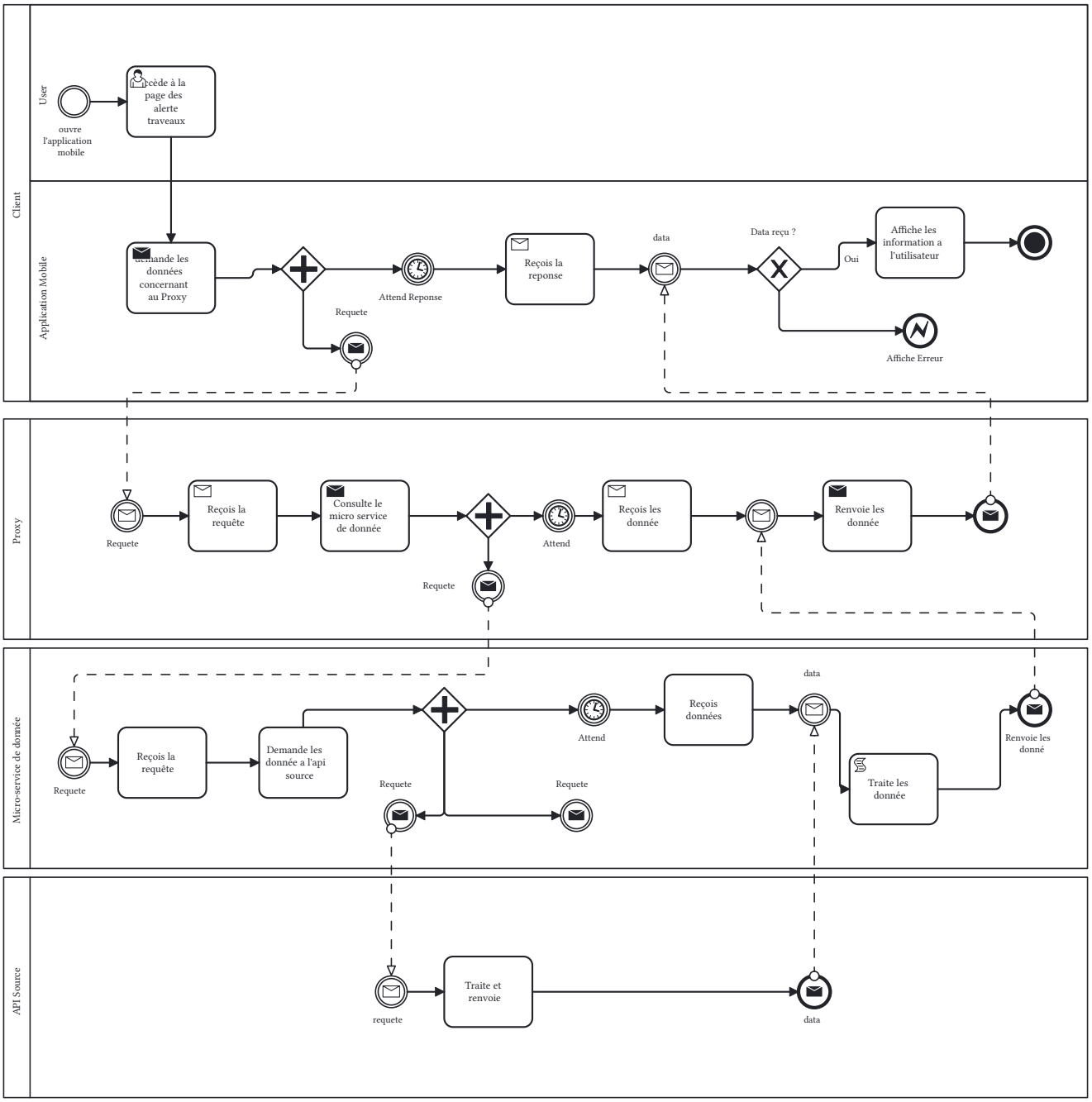
Figure 6 — Affichage tutoriel Bicloo



L'utilisateur ouvre l'application puis accède à la page du tutoriel. L'application affiche directement le tutoriel d'utilisation et termine le processus, sans échange avec un proxy ni appel à une API externe.

IV.7. Consultation alertes travaux

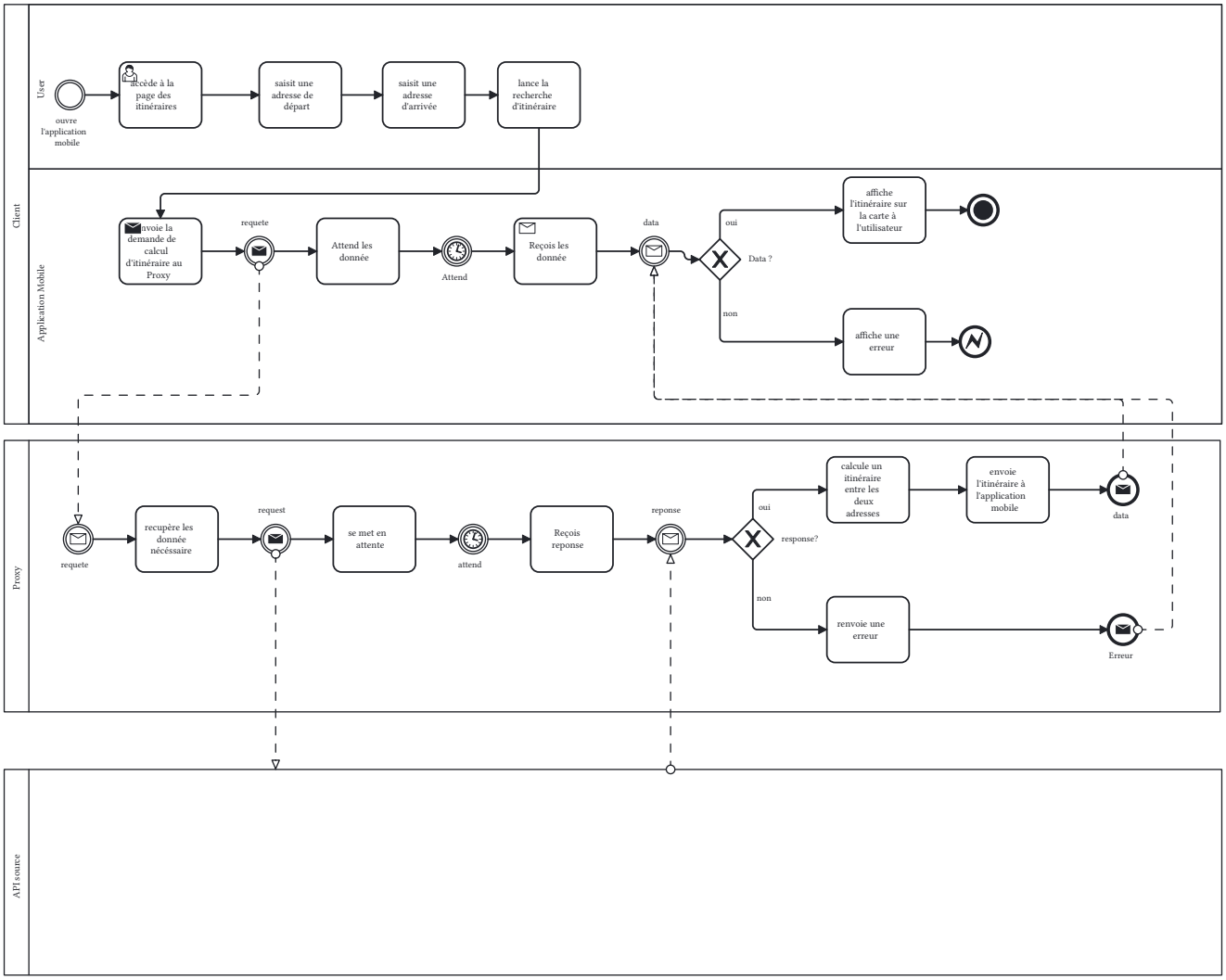
Figure 7 — Affichage alertes travaux



Le scénario démarre quand l'utilisateur ouvre l'application puis accède à la page des alertes travaux. L'application envoie la requête au proxy et attend. Le proxy transmet la demande à un micro-service de données, qui interroge l'API source, attend le retour, traite les informations puis les renvoie au proxy. Le proxy renvoie ensuite les données à l'application, qui affiche les alertes si des données sont reçues, sinon affiche une erreur.

IV.8. Consultation itinéraire

Figure 8 — Affichage itinéraire vélo



L'utilisateur ouvre la page des itinéraires, saisit une adresse de départ, une adresse d'arrivée, puis lance la recherche. L'application envoie une requête de calcul au proxy et se met en attente. Le proxy récupère d'abord les données nécessaires, sollicite l'API source, attend la réponse, puis vérifie sa validité: si la réponse est correcte, il calcule l'itinéraire, l'envoie à l'application et celle-ci l'affiche sur la carte ; sinon, le proxy renvoie une erreur et l'application affiche un message d'échec.